

Curso: Programación Avanzada

Guía: 18

Título: GUARDS E INTERCEPTORES EN ANGULAR

Duración estimada: 180 minutos

Docentes: Christian Andrés Candela

Objetivo

Implementar la capa de seguridad del frontend Angular del proyecto de solicitudes universitarias: proteger rutas con **Guards** funcionales y añadir el token JWT a las peticiones HTTP con un **Interceptor** funcional, siguiendo los patrones Angular 21 establecidos en las Guías 15, 16 y 17.

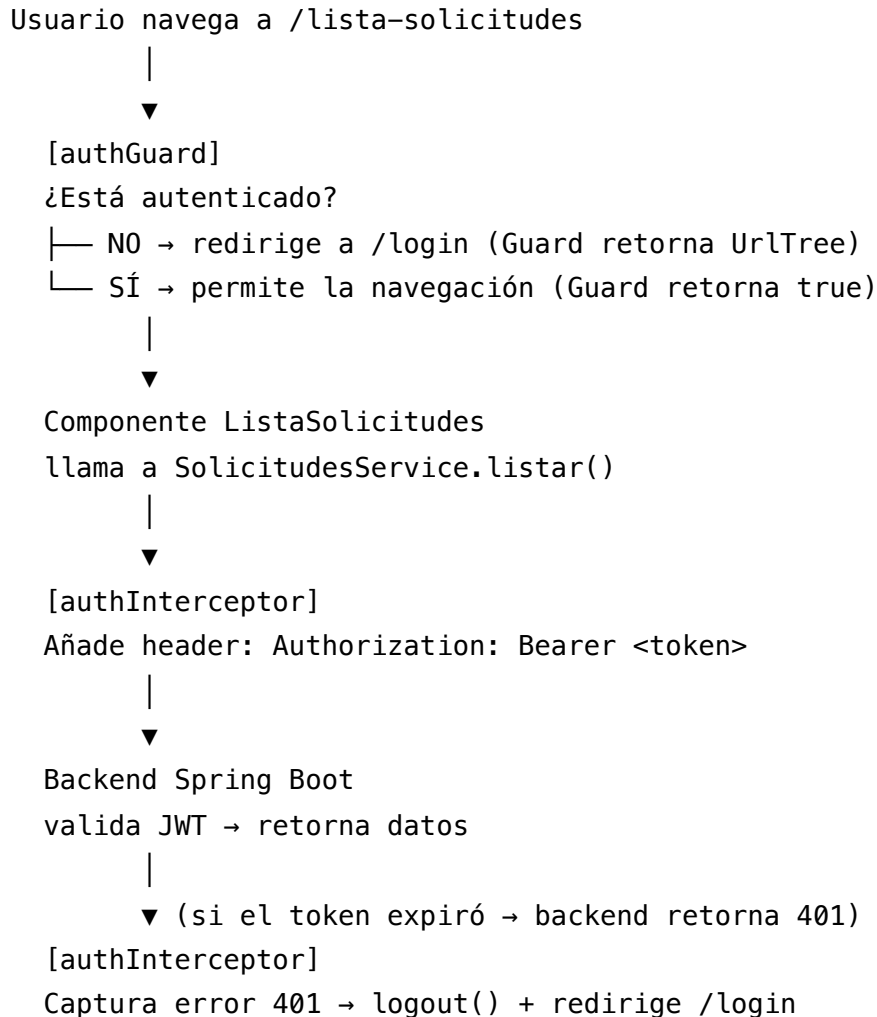
Conceptos Básicos Previos

- **Guard:** Función que Angular ejecuta antes de activar una ruta. Decide si la navegación continúa o se redirige.
- **Interceptor HTTP:** Función que intercepta todas las peticiones HTTP salientes (y las respuestas entrantes) para modificarlas de forma centralizada.
- **JWT (JSON Web Token):** Token de autenticación que el backend retorna al hacer login. Debe enviarse en el header `Authorization: Bearer <token>` en cada petición protegida.
- **CanActivateFn :** Tipo funcional de guard en Angular (desde Angular 15). Reemplaza la clase `CanActivate` (legacy).
- **HttpInterceptorFn :** Tipo funcional de interceptor (desde Angular 15). Reemplaza la clase `HttpInterceptor` (legacy).
- **route.data :** Propiedad de la definición de una ruta que permite pasar metadata estática al guard o componente.

Contextualización Teórica

◆ Flujo de Seguridad Completo

Antes de escribir código, es fundamental entender cómo interactúan los Guards y el Interceptor en el ciclo de vida de una navegación:



◆ Guards Funcionales — CanActivateFn

Desde Angular 15, los guards se definen como **funciones** (CanActivateFn), no como clases. Esto es más simple, testeable y alineado con la filosofía standalone de Angular 21:

```
// ✅ Functional Guard (Angular 15+) – patrón moderno
export const authGuard: CanActivateFn = (route, state) => {
  const auth = inject(AuthService);
  return auth.isAuthenticated() // lee el signal
    ? true
    : inject(Router).createUrlTree(['/login']);
};

// ❌ Class Guard (legacy, Angular < 15) – ya no recomendado
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  canActivate(): boolean { ... }
}
```

Valores de retorno de un Guard:

Retorno	Efecto
true	La navegación continúa
false	La navegación se cancela (URL no cambia)
UrlTree (de router.createUrlTree)	Redirige a otra ruta

createUrlTree vs navigate : Un guard debe **retornar** una `UrlTree` en lugar de llamar `router.navigate()` directamente. Esto permite que Angular gestione la navegación de forma predecible y no cause dobles navegaciones.

💎 Tipos de Guards en Angular 21

Guard	Cuándo se ejecuta	Uso típico
CanActivateFn	Antes de activar una ruta	Verificar autenticación/roles
CanDeactivateFn	Antes de abandonar una ruta	Confirmar si hay cambios sin guardar
CanMatchFn	Antes de cargar el módulo/ruta	Filtrar rutas por condición
ResolveFn	Antes de activar, carga datos	Pre-cargar datos para el componente

En esta guía implementamos `CanActivateFn` .

◆ Interceptores Funcionales — `HttpInterceptorFn`

El interceptor es una **función** que recibe la petición (`req`) y una función `next` que la procesa. Puede modificar la petición antes de enviarla y la respuesta al recibirla:

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {  
  // 1. Modificar la petición saliente  
  const modifiedReq = req.clone({ setHeaders: { Authorization: 'Bearer token' } });  
  
  // 2. Pasar la petición (modificada o no) al siguiente interceptor o al backend  
  return next(modifiedReq).pipe(  
    // 3. Interceptar la respuesta entrante (opcional)  
    catchError(error => { ... })  
  );  
};
```

El interceptor funcional usa `inject()` directamente — no necesita constructor ni `@Injectable`.

◆ `route.data` — Metadata en Rutas

La propiedad `data` de una ruta permite pasar información estática al guard o al componente sin usar un servicio:

```
{  
  path: 'admin',  
  component: AdminPanel,  
  canActivate: [rolesGuard],  
  data: { expectedRoles: ['ADMIN'] } // ← metadata estática  
}  
  
// En el guard:  
const expectedRoles: string[] = route.data['expectedRoles'] || [];
```

⚠ Precauciones y Recomendaciones

- Asegúrate de tener el proyecto del backend corriendo en `http://localhost:8080`.

- Los Guards y el Interceptor de esta guía dependen del `AuthService` de Guías 15-17. Verifica que `isAuthenticated` es un `WritableSignal` y que `getToken()` retorna `localStorage.getItem('token')`.
- El interceptor `authInterceptor` fue referenciado en `app.config.ts` de las Guías 15 y 17. Si ya lo creaste, en esta guía solo añades el manejo de error 401.



Artefactos Requeridos

- Proyecto Angular con Guards mínimos y login funcionando (Guías 15–17).
- Backend Spring Boot corriendo y autenticando con JWT.



Evaluación o Resultado Esperado

Se espera que el estudiante:

1. Implemente un `authGuard` que proteja `/lista-solicitudes` y `/nueva-solicitud`.
2. Implemente un `publicGuard` que evite que usuarios autenticados accedan a `/login`.
3. Implemente un `rolesGuard` para proteger rutas según rol.
4. Tenga un `authInterceptor` que añade el token JWT y maneja errores 401.

Procedimiento

1. `authGuard` — Proteger rutas que requieren autenticación

El `authGuard` protege rutas que solo deben ser accesibles por usuarios autenticados. Si el usuario no tiene sesión, lo redirige a `/login`.

1.1 Genera el guard:

```
ng generate guard guards/auth --skip-tests
```

Cuando Angular CLI pregunte qué interfaz implementar, selecciona `CanActivate`. Luego reemplazaremos el contenido.

1.2 Implementa auth.guard.ts :

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../servicios/auth';

/**
 * Protege rutas que requieren autenticación.
 * Si el usuario NO está autenticado → redirige a /login.
 * Si el usuario SÍ está autenticado → permite el acceso.
 */
export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // ✅ Lee el signal de autenticación (mismo signal que usa el Header)
  return authService.isAuthenticated()
    ? true
    : router.createUrlTree(['/login']);
};
```

¿Por qué `createUrlTree` en lugar de `router.navigate` ? Los Guards deben **retornar** la redirección como valor, no ejecutarla directamente. `createUrlTree` crea un objeto de navegación que Angular procesa de forma controlada, evitando dobles navegaciones y efectos secundarios.

2. publicGuard — Evitar que usuarios autenticados accedan al login

Un usuario que ya inició sesión no debería poder navegar a `/login` . El `publicGuard` lo redirige a `/lista-solicitudes` en ese caso.

2.1 Genera el guard:

```
ng generate guard guards/public --skip-tests
```

2.2 Implementa `public.guard.ts` :

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../servicios/auth';

/**
 * Protege rutas públicas (login).
 * Si el usuario YA está autenticado → redirige a /lista-solicitudes.
 * Si NO está autenticado → permite el acceso.
 */
export const publicGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  return authService.isAuthenticated()
    ? router.createUrlTree(['/lista-solicitudes'])
    : true;
};
```

Nota la **lógica inversa** respecto a `authGuard` : aquí `isAuthenticated() === true` **bloquea** el acceso a la ruta pública.

3. Aplicar Guards a las Rutas del Proyecto

Actualiza `app.routes.ts` con los guards correctos:

```

import { Routes } from '@angular/router';
import { Login } from './componentes/login/login';
import { NuevaSolicitud } from './componentes/nueva-solicitud/nueva-solicitud';
import { ListaSolicitudes } from './componentes/lista-solicitudes/lista-solicitudes';
import { authGuard } from './guards/auth.guard';
import { publicGuard } from './guards/public.guard';

export const routes: Routes = [
  // Ruta pública – usuarios autenticados son redirigidos a /lista-solicitudes
  { path: 'login', component: Login, canActivate: [publicGuard] },

  // Rutas protegidas – requieren autenticación
  { path: 'lista-solicitudes', component: ListaSolicitudes, canActivate: [authGuard] },
  { path: 'nueva-solicitud', component: NuevaSolicitud, canActivate: [authGuard] },

  // Ruta comodín
  { path: '**', pathMatch: 'full', redirectTo: '/' },
];

```

3.1 Ejecuta y verifica:

npm start

1. Sin iniciar sesión, intenta navegar a `http://localhost:4200/lista-solicitudes` → debe redirigir a `/login`.
2. Inicia sesión e intenta navegar a `http://localhost:4200/login` → debe redirigir a `/lista-solicitudes`.
3. Inicia sesión y navega a `http://localhost:4200/lista-solicitudes` → debe cargar correctamente.

4. rolesGuard — Proteger rutas por Rol

Para el proyecto final, es posible que existan rutas solo accesibles para administradores. El `rolesGuard` combina verificación de autenticación y verificación de rol.

4.1 Genera el guard:

ng generate guard guards/roles --skip-tests

4.2 Genera el componente de acceso no autorizado:

```
ng generate component componentes/authorized --skip-tests
```

4.3 Implementa roles.guard.ts :

```
import { CanActivateFn, Router } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../servicios/auth';

/**
 * Protege rutas que requieren un rol específico.
 * 1. Si NO está autenticado → redirige a /login.
 * 2. Si está autenticado pero NO tiene el rol → redirige a /unauthorized.
 * 3. Si tiene el rol → permite el acceso.
 *
 * Uso en rutas:
 * { path: 'admin', canActivate: [rolesGuard], data: { expectedRoles: ['ADMIN'] } }
 */
export const rolesGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  // Paso 1: verificar autenticación
  if (!authService.isAuthenticated()) {
    return router.createUrlTree(['/login']);
  }

  // Paso 2: verificar rol
  // route.data['expectedRoles'] viene de la definición de la ruta
  const expectedRoles: string[] = route.data['expectedRoles'] || [];

  // Si no se especifican roles requeridos, solo se exige autenticación
  if (expectedRoles.length === 0) return true;

  const userRoles: string[] = authService.getRoles();
  const hasRole = expectedRoles.some(role => userRoles.includes(role));

  return hasRole ? true : router.createUrlTree(['/unauthorized']);
};
```

4.4 Añade getRoles() al AuthService (si no existe):

```
// En src/app/servicios/auth.ts
getRoles(): string[] {
  const roles = localStorage.getItem('roles');
  return roles ? JSON.parse(roles) : [];
}
```

4.5 Añade las rutas de administrador en app.routes.ts :

```
import { rolesGuard }    from './guards/roles.guard';
import { Unauthorized }   from './componentes/unauthorized/unauthorized';

export const routes: Routes = [
  // ... rutas existentes ...

  // Ruta de administrador — requiere rol ADMIN
  { path: 'admin', component: /* tu componente admin */, canActivate: [rolesGuard],
    data: { expectedRoles: ['ADMIN'] } },

  // Página de acceso no autorizado
  { path: 'unauthorized', component: Unauthorized },
];
```

route.data : el objeto `data` es estático — se define al crear la ruta y no cambia en tiempo de ejecución. En el guard se accede con `route.data['expectedRoles']` . Esto desacopla los roles del guard del componente: el mismo `rolesGuard` sirve para cualquier ruta, solo cambia el valor de `data` .

4.6 Ejecuta y verifica:

```
npm start
```

1. Inicia sesión con un usuario `USER` e intenta navegar a `http://localhost:4200/admin` → debe redirigir a `/unauthorized` .
2. Inicia sesión con un usuario `ADMIN` → debe poder acceder a `/admin` .

5. authInterceptor — Añadir el Token JWT a las Peticiones

El interceptor captura **todas** las peticiones HTTP salientes de la aplicación y añade el header `Authorization: Bearer <token>` cuando el usuario está autenticado. También maneja el error 401 (token expirado) redirigiendo al login.

Nota: Si ya creaste el interceptor en la Guía 15, solo necesitas añadir el bloque `catchError` para el manejo del 401. El resto ya existe.

5.1 Crea o actualiza `src/app/interceptores/auth.interceptor.ts` :

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject }           from '@angular/core';
import { Router }           from '@angular/router';
import { catchError, throwError } from 'rxjs';
import { AuthService } from '../servicios/auth';

/**
 * Interceptor de autenticación JWT.
 *
 * Para cada petición HTTP:
 * 1. Si el usuario está autenticado, añade el header Authorization.
 * 2. Si el backend responde con 401 (token expirado), hace logout y redirige al login.
 */
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authService = inject(AuthService);
  const router      = inject(Router);

  // Si no está autenticado, pasa la petición sin modificar
  // (el guard ya protege las rutas, pero las peticiones de login no deben llevar token
  if (!authService.isAuthenticated()) {
    return next(req);
  }

  const token = authService.getToken();

  // Clona la petición original añadiendo el header de autorización
  // req es inmutable – siempre se usa clone() para modificarlo
  const authReq = req.clone({
    headers: {
      Authorization: `Bearer ${token}`
    }
  });

  return next(authReq).pipe(
    catchError(error => {
      // Token expirado o inválido – cerrar sesión y redirigir
      if (error.status === 401) {
        authService.logout(); // limpia localStorage y signal
        router.navigate(['/login']);
      }
      return throwError(() => error);
    })
  );
}
```

```

    })
  );
};

```

¿Por qué `req.clone()` ? Las peticiones HTTP en Angular son **inmutables**. No se puede modificar `req` directamente. `clone()` crea una copia con los cambios aplicados, dejando la original intacta.

¿Por qué verificar `isAuthenticated()` antes de añadir el token? La petición de login (`POST /api/auth/login`) no debe llevar un token — no existe aún. Al verificar el estado de autenticación, el interceptor omite añadir el header en peticiones públicas.

6. Registrar el Interceptor en `app.config.ts`

```

import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideAnimationsAsync } from '@angular/platform-browser/animations/async';
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { providePrimeNG } from 'primeng/config';
import { Aura } from 'primeng/themes/aura';
import { routes } from './app.routes';
import { authInterceptor } from './interceptores/auth.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor])), // ← interceptor registrad
    provideAnimationsAsync(),
    providePrimeNG({
      theme: {
        preset: Aura,
        options: { darkModeSelector: '.app-dark' }
      }
    })
  ]
};

```

`withInterceptors([...])` acepta un array — si en el futuro necesitas múltiples interceptores (logging, manejo de errores global), simplemente añádelos al array. Se ejecutan en orden.

7. Pruebas Finales

```
npm start
```

Ejecuta estas pruebas de forma ordenada:

#	Acción	Resultado esperado
1	Sin sesión, navega a <code>/lista-solicitudes</code>	Redirige a <code>/login</code>
2	Sin sesión, navega a <code>/nueva-solicitud</code>	Redirige a <code>/login</code>
3	Ya autenticado, navega a <code>/login</code>	Redirige a <code>/lista-solicitudes</code>
4	Inicia sesión con usuario <code>USER</code>	Redirige a <code>/lista-solicitudes</code> , header muestra menú autenticado
5	Con sesión <code>USER</code> , navega a <code>/admin</code>	Redirige a <code>/unauthorized</code>
6	En DevTools → Network, revisa la petición a <code>/api/solicitudes</code>	Debe tener header <code>Authorization: Bearer <token></code>
7	Modifica manualmente el token en <code>localStorage</code> a un valor inválido	Al navegar, el backend retorna 401, el interceptor hace logout y redirige a <code>/login</code>
8	Cierra sesión (botón Logout del Header)	El signal <code>isAuthenticated</code> se pone en <code>false</code> , el Header muestra botón <code>Ingresar</code>

Verificar el token en DevTools:

1. Abre las Herramientas de Desarrollador (F12) → pestaña **Network**.
2. Filtra por `XHR` o `Fetch` .
3. Inicia sesión y navega a `/lista-solicitudes` .
4. Selecciona la petición a `/api/solicitudes` → pestaña **Headers**.

5. Busca el header `Authorization: Bearer ...` — confirma que el token está presente.

8. 📦 Control de Versiones

```
git add .
git commit -m "feat: guards (auth, public, roles) e interceptor JWT"
git push origin main
```

6. Avance del Proyecto Final

1. Aplica `authGuard` a todas las rutas privadas de tu proyecto.
2. Aplica `publicGuard` a la ruta de login.
3. Si tu proyecto tiene roles (admin vs usuario), aplica `rolesGuard` con los roles correspondientes.
4. Verifica en DevTools que el token se envía correctamente en todas las peticiones protegidas.

🔍 Solución de Problemas

- **La ruta protegida carga sin estar autenticado:**
 - Verifica que el guard está importado correctamente en `app.routes.ts`.
 - Verifica que `canActivate: [authGuard]` está en la definición de la ruta.
- **El interceptor no añade el token:**
 - Verifica que `withInterceptors([authInterceptor])` está en `provideHttpClient` en `app.config.ts`.
 - Verifica que `authService.getToken()` retorna `localStorage.getItem('token')` (clave `'token'`).
- **Error: inject() must be called from an injection context :**
 - `inject()` solo puede usarse dentro de un contexto de inyección (constructor, función de factory, guard funcional). Verifica que el guard es una `CanActivateFn`, no una función normal.
- **El guard redirige en bucle infinito:**
 - Verifica que la ruta `/login` tiene `publicGuard` (no `authGuard`).
 - Verifica que la ruta `/lista-solicitudes` tiene `authGuard` (no `publicGuard`).

- **Error 401 en la petición de login:**
 - El interceptor no debe añadir el token a la petición de login. Verifica que la condición `if (!authService.isAuthenticated())` está presente.

Preguntas para Reflexión

- ¿Por qué se usa `router.createUrlTree(['/login'])` en lugar de `router.navigate(['/login'])` en un guard?
- ¿Qué diferencia hay entre `authGuard` y `publicGuard` ? ¿Por qué se necesitan ambos?
- ¿Por qué el interceptor usa `req.clone()` en lugar de modificar `req` directamente?
- ¿Qué pasa si hay múltiples interceptores registrados en `withInterceptors` ? ¿En qué orden se ejecutan?
- ¿Por qué es importante que el guard lea `authService.isAuthenticated()` como signal y no como método que consulta `localStorage` en cada llamada?

Referencias Bibliográficas

- [Angular Route Guards](#)
- [Angular](#) CanActivateFn
- [Angular HTTP Interceptors](#)
- [Angular](#) HttpInterceptorFn
- [JWT Introduction](#)
- [Angular](#) inject()